

Wearable Tech Lab · Full Curriculum

Single session (2.5 hrs) · 9–13 yrs · Beginner → Intermediate

Full facilitator curriculum **PREMIUM**

Overview

Kids design and build a wearable gadget they wire, code and wear home.

DETAIL	
Track	Arduino Robotics & Electronics
Format	Single session (2.5 hrs)
Ages	9–13 yrs
Level	Beginner → Intermediate
Price	from KES 5,000
Standalone	Yes
Prerequisites	None — standalone

Course Overview

Wearable Tech Lab is a single, intensive 2.5-hour workshop where each student builds a fully working LED wearable — a wristband or cap fitted with an addressable NeoPixel strip, a sound sensor, a mode button and an Arduino Nano brain — and leaves wearing it home, battery powered. The course compresses electronics assembly, C++ programming and creative design into one tight arc: students first wire up their hardware and prove every connection works, then they program three distinct light behaviours (a rotating rainbow, a pulsing "heartbeat", and a mode that reacts to noise), and finally they personalise the physical wearable and add their own signature colour mode to the code.

The session is deliberately sequenced so hardware problems are solved before software is layered on top — nothing is more frustrating than debugging code on a wearable with a loose wire. By the time programming starts, every student's circuit

has already been verified with a wiring-test sketch, so all classroom time in the coding block is spent on logic, not soldering.

The take-home is a complete, four-mode LED wearable (rainbow cycle, heartbeat pulse, sound-reactive colour, and a personalised signature-colour mode), decorated to the student's own taste, powered by a small USB power bank so it works the moment they walk out the door.

Learning Outcomes

- Wire and test an addressable NeoPixel (WS2812B) LED strip, including correct use of an inline resistor and decoupling capacitor.
- Read analog sensor data (a sound/microphone module) and use `map()` and `constrain()` to convert raw readings into usable light output.
- Write and upload Arduino sketches using the Adafruit_NeoPixel library, including HSV colour, brightness control and simple animation timing without blocking hardware responsiveness.
- Implement a debounced push-button state machine to cycle between multiple program "modes" on one device.
- Diagnose and fix common wiring and code faults using the Serial Monitor as a debugging tool.
- Apply basic craft/textile techniques (gluing, mounting, wire management) to integrate electronics into a wearable product safely and durably.
- Present a finished technical project clearly, explaining design choices and demonstrating functionality live.

Materials Master List

ITEM	SPEC	QTY PER STUDENT/ GROUP	NOTES
Arduino Nano (or compatible clone)	ATmega328P, Micro-USB port	1 per student	Facilitator should pre-test each board with the blink example before the session
USB power bank	5V output, 1000–2000 mAh, Micro-USB	1 per student	Take-home power source; charge all banks the night before
Micro-USB cable		1 per student	

	Short, 20–30 cm, data-capable (not charge-only)		Used for both programming and later power-bank connection
NeoPixel WS2812B flexible LED strip	5V, 60 LEDs/m reel, cut to 8-LED segments with solder pads	1 segment (8 LEDs) per student	Pre-cut by facilitator along the marked copper cut-lines before the session; source by the metre from Pixel Electronics or Nerokas
Sound sensor module	KY-038 style, onboard mic + comparator, AO and DO pins, onboard sensitivity potentiometer	1 per student	Analog output (AO) is used in this course
Tactile push button	6 mm, 4-leg through-hole	1 per student	Used as the mode-switch button
Resistor	330–470 Ω , 1/4 W	1 per student	Inline on the NeoPixel data line, protects the first LED
Electrolytic capacitor	1000 μ F, 6.3 V or higher	1 per student	Across NeoPixel 5V/GND, smooths power spikes
Pre-tinned stranded wire or alligator-clip leads	26 AWG silicone wire, assorted colours, or clip-to-clip leads 15 cm	8 per student	Solder-free connections; colour-code power/ground/signal
Mini solderless breadboard	170-point	1 per student	Optional staging area for testing connections before final mounting
Heat-shrink tubing	Assorted diameters, 3–5 cm pieces	6 per student	Insulates exposed joints once wiring is confirmed working
Fabric wristband or cap blank	Velcro-strap felt/neoprene wristband (3–4 cm wide) OR plain baseball cap	1 per student	Student chooses wristband or cap at the start of Session 1
Hot glue gun + glue sticks	Low-temp mini glue gun	1 gun per 4 students, 3 sticks per student	Low-temp reduces burn risk for this age group
Fabric glue (fallback)	Washable fabric glue, small bottle	1 bottle per 4 students	Alternative to hot glue for delicate fabrics
Conductive thread (optional sewing path)	Stainless steel, conductive, ~2 ply	2 m + 1 embroidery needle per student opting to sew	Only for students choosing to sew rather than glue/clip the strip
Craft decoration pack	Felt scraps, sequins, ribbon, buttons, googly eyes, fabric	1 assorted set per 2 students	For Session 3 personalisation

paint pens, permanent markers

Small pouch or heat-shrink Nano housing	Clear vinyl pouch or 3D-printed clip case	1 per student	Protects Nano and power bank on the wearable
Double-sided Velcro coins	2 cm diameter, self-adhesive	4 per student	Mounts Nano/power bank to band or cap without permanent glue
Masking tape + fine marker	—	1 roll + 1 marker per 4 students	Labelling wires and parts during wiring
Multimeter	Basic digital, continuity + voltage	1 per 4 students	Facilitator-led troubleshooting tool
Laptop with Arduino IDE 2.x installed	Adafruit_NeoPixel library pre-installed	1 per student (or pair)	Install library beforehand: Sketch → Include Library → Manage Libraries → "Adafruit NeoPixel"
Spare Nanos, strips, sensors	Same spec as above	10% extra of each	Contingency for dead-on-arrival components

Session Plans

Part 1 — Build: Wire the Wearable

Objectives

- Students correctly wire a NeoPixel strip, sound sensor and push button to an Arduino Nano.
- Students physically mount the strip and components onto a wristband or cap base.
- Students verify every connection using a pre-written wiring-test sketch before moving to programming.

Timing (45 minutes)

- 0:00-0:05 — Welcome, safety briefing (glue gun heat, careful handling of the cut LED strip edges, no unplugging while powered)
- 0:05-0:12 — Show a finished exemplar wearable running all four modes; explain today's build plan and the wiring diagram on the board
- 0:12-0:15 — Distribute kits, students choose wristband or cap base

- 0:15-0:35 — Guided hands-on wiring: strip → Nano, sound sensor → Nano, button → Nano, then temporary mounting of strip onto the band/cap with a small dab of glue or Velcro (not final decoration yet)
- 0:35-0:42 — Upload the wiring-test sketch from a facilitator laptop or student laptop; each student confirms colour wipe, Serial sound readings, and button response
- 0:42-0:45 — Quick fixes for anyone with a fault; tidy loose wires with tape ahead of the programming block

Step-by-step

1. Open with the safety briefing — call out the glue gun nozzle, sharp scissors (if trimming strip), and the rule "never unplug or replug a wire while the Nano has power."
2. Hold up the exemplar wearable, cycle through its modes with the button so students see the goal before they start.
3. Draw the wiring diagram on the board (strip DIN→D6 via resistor, strip 5V/GND, sound sensor AO→A0, VCC/GND, button legs→D7 and GND). Read it aloud with the class once.
4. Hand out kits. Have students identify each component before touching a wire: "hold up your NeoPixel strip... hold up your sound sensor... hold up your button."
5. Walk the room as students make connections one at a time, checking each connection against the diagram before they move to the next.
6. Once wired, help students temporarily attach the strip to their band/cap using a small dot of hot glue or a Velcro coin — just enough to hold it still for testing, not the final permanent mount (that happens in Part 3).
7. Load the wiring-test sketch (pre-saved on facilitator laptop or handed out as a text file) onto each Nano via USB.
8. Have students open the Serial Monitor, clap near their sound sensor and watch the printed number jump, then press their button and watch the strip turn white.
9. Fix any non-working component immediately using the multimeter (continuity test on suspect wires) before moving to Part 2 — do not let unresolved hardware faults carry into the coding session.

Build detail

Materials: Arduino Nano, 8-LED NeoPixel strip segment, sound sensor module, tactile button, 330-470 Ω resistor, 1000 μ F capacitor, 8 pre-tinned wires/alligator

leads, wristband or cap blank, hot glue gun, Velcro coins, multimeter, laptop with Arduino IDE.

Wiring/assembly list (component → pin):

- NeoPixel strip DIN (data in) → Arduino Nano **D6**, in series through the 330–470 Ω resistor
- NeoPixel strip 5V (V+) → Nano **5V**
- NeoPixel strip GND → Nano **GND** (shared ground rail with all other components)
- 1000 μ F capacitor: positive leg to the strip's 5V line, negative leg to GND, placed as close to the strip's input end as possible
- Sound sensor VCC → Nano **5V**
- Sound sensor GND → Nano **GND**
- Sound sensor AO (analog out) → Nano **A0**
- Push button leg 1 → Nano **D7**
- Push button leg 2 → Nano **GND**
- Physical mount: NeoPixel strip laid along the outside of the wristband/cap brim, sound sensor mic facing outward for good pickup, button positioned where a thumb can reach it, Nano tucked into the pouch with its USB port left accessible

```
#include <Adafruit_NeoPixel.h>

#define LED_PIN    6      // NeoPixel data line (through resistor)
#define LED_COUNT  8      // Number of LEDs in the wearable strip
#define BUTTON_PIN  7      // Mode button, other leg to GND
#define SOUND_PIN  A0      // Sound sensor analog output

Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
  strip.begin();           // Start the NeoPixel library
  strip.show();            // Make sure all pixels start OFF
  strip.setBrightness(60); // Moderate brightness for testing
  pinMode(BUTTON_PIN, INPUT_PULLUP); // Button reads LOW when pressed
  Serial.begin(9600);
  Serial.println("Wiring test starting...");
}

void loop() {
  // 1) Check the LED strip wiring - colour wipe test
  colorWipe(strip.Color(255, 0, 0), 80); // Red
  colorWipe(strip.Color(0, 255, 0), 80); // Green
  colorWipe(strip.Color(0, 0, 255), 80); // Blue

  // 2) Check the sound sensor wiring - print reading to Serial Monitor
  int soundLevel = analogRead(SOUND_PIN);
```

```

Serial.print("Sound sensor reading: ");
Serial.println(soundLevel);    // Should jump up when you clap or talk loudly

// 3) Check the button wiring - strip goes white while button is held
if (digitalRead(BUTTON_PIN) == LOW) {
  colorWipe(strip.Color(255, 255, 255), 0);
  Serial.println("Button pressed!");
} else {
  colorWipe(strip.Color(0, 0, 0), 0);
}

delay(200);
}

// Fills the strip one LED at a time with the given colour
void colorWipe(uint32_t color, int wait) {
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, color);
    strip.show();
    if (wait > 0) delay(wait);
  }
}

```

Checks for understanding

- "Why does the NeoPixel data line need a resistor before it reaches D6?"
- "What would you see on the strip if the GND wire from the sound sensor wasn't connected?"
- "How does the Nano know the button has been pressed rather than left alone?"

MISTAKE	FIX
Strip shows no light at all	Check 5V and GND are not swapped, confirm resistor is on the DIN line not the power line, re-seat wire into D6
Only the first LED lights, rest stay dark	Check the strip was cut cleanly on the marked line and the DIN pad wasn't torn; try a spare segment
Serial Monitor shows constant 0 or 1023 from sound sensor	Turn the onboard potentiometer on the sensor slowly while watching the monitor to recentre sensitivity
Button always reads pressed or never reads pressed	Confirm INPUT_PULLUP is set and the second leg truly goes to GND, not 5V

Part 2 — Program: Light Patterns

Objectives

- Students write and upload code producing a rainbow cycle, a heartbeat pulse, and a sound-reactive colour mode.

- Students implement a debounced button to cycle between the three modes on one device.
- Students use the Serial Monitor to calibrate the sound sensor's response.

Timing (80 minutes)

- 0:00–0:08 — Confirm Arduino IDE and Adafruit_NeoPixel library are ready on every laptop; quick recap of `setup()/loop()` structure
- 0:08–0:15 — Explain how NeoPixel colour works: RGB vs HSV, `strip.setPixelColor()`, `strip.show()`
- 0:15–0:30 — Live-code the rainbow cycle function together, upload, everyone sees their strip rotate colour
- 0:30–0:45 — Live-code the heartbeat pulse (fade brightness up and down), upload, compare pulse speed choices
- 0:45–0:60 — Explain analog sound mapping; live-code the sound-react function; calibrate each sensor's potentiometer while clapping/talking
- 0:60–0:75 — Combine all three into one sketch with a debounced button state machine; upload and test full mode-cycling
- 0:75–0:80 — Roaming debug time; facilitator checks every wearable cycles cleanly through all three modes

Step-by-step

1. Open the IDE together and confirm Tools → Board is set to Arduino Nano and the correct COM/serial port is selected on every laptop.
2. Explain `strip.Color(r,g,b)` for direct RGB and `strip.ColorHSV(hue,sat,val)` for smooth colour rotation — hue runs 0–65535 in the library, not 0–360.
3. Type the rainbow cycle function together on the board, explaining that `firstPixelHue` increases a little each loop to slowly rotate the colours around the strip.
4. Upload and celebrate — every wristband should now show a moving rainbow.
5. Introduce the heartbeat concept: brightness ramps up, hits a peak, then ramps back down, using a variable that flips direction (`heartStep` reversing sign) instead of a fixed sequence.
6. Upload, ask students to compare: "does yours pulse like a real heartbeat or too fast/slow?" and adjust `heartStep` together.

7. Introduce `analogRead()` and `map()`: show how a raw 0-1023 sound reading becomes a hue (quiet=blue, loud=red) and a brightness value.
8. Have every student open Serial Monitor, clap, and read their own numbers, then adjust the sensor's onboard potentiometer if readings are too flat or too jumpy.
9. Now show how a single button can switch between the three "loop" behaviours by storing a `mode` variable and using a `switch` statement, with a small debounce function so one press doesn't register as five presses.
10. Upload the full combined sketch, test button presses cycle 0→1→2→0 correctly, celebrate working wearables.

Build detail

Materials: same hardware as Part 1 (no new components), laptops with Arduino IDE, Adafruit_NeoPixel library installed.

Wiring/assembly list: unchanged from Part 1 — this session is pure software, no rewiring needed. Facilitator should only re-check that no wires worked loose during handling.

```
#include <Adafruit_NeoPixel.h>

#define LED_PIN    6      // NeoPixel strip data pin
#define LED_COUNT  8      // Number of LEDs on the wearable
#define BUTTON_PIN  7      // Mode button (other leg to GND)
#define SOUND_PIN  A0      // Sound sensor analog output

Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

int mode = 0;              // 0 = Rainbow, 1 = Heartbeat, 2 = Sound React
const int NUM_MODES = 3;

// Button debounce variables
int lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;

// Heartbeat variables
float heartBrightness = 0;
float heartStep = 6;

void setup() {
  strip.begin();
  strip.show();
  strip.setBrightness(70);
  pinMode(BUTTON_PIN, INPUT_PULLUP); // Button reads LOW when pressed
  Serial.begin(9600);                // Watch sound sensor readings here
}

void loop() {
  checkButton();                    // See if the student wants to switch modes
```

```

switch (mode) {
  case 0: rainbowCycle(10); break;
  case 1: heartbeatPulse(); break;
  case 2: soundReact(); break;
}
}

// ----- MODE 0: Rainbow Cycle -----
void rainbowCycle(int waitMs) {
  static long firstPixelHue = 0;
  for (int i = 0; i < strip.numPixels(); i++) {
    int pixelHue = firstPixelHue + (i * 65536L / strip.numPixels());
    strip.setPixelColor(i, strip.gamma32(strip.ColorHSV(pixelHue)));
  }
  strip.show();
  firstPixelHue += 256;          // Slowly rotate the rainbow each loop
  if (firstPixelHue >= 65536) firstPixelHue = 0;
  delay(waitMs);
  checkButton();                // Stay responsive to the button mid-animation
}

// ----- MODE 1: Heartbeat Pulse -----
void heartbeatPulse() {
  heartBrightness += heartStep;
  if (heartBrightness >= 255 || heartBrightness <= 0) {
    heartStep = -heartStep;      // Reverse direction: fade in / fade out
  }
  heartBrightness = constrain(heartBrightness, 0, 255);

  uint32_t red = strip.Color((int)heartBrightness, 0, 0);
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, red);
  }
  strip.show();
  delay(15);
  checkButton();
}

// ----- MODE 2: Colour React to Sound -----
void soundReact() {
  int soundLevel = analogRead(SOUND_PIN);    // 0-1023 from the sound sensor
  Serial.println(soundLevel);                // Watch this to calibrate sensitivity

  // Map loudness to a hue: quiet = blue, loud = red
  int hue = map(soundLevel, 0, 800, 43690, 0);
  hue = constrain(hue, 0, 65535);

  // Map loudness to brightness too, so silence dims the strip
  int brightness = map(soundLevel, 0, 800, 20, 255);
  brightness = constrain(brightness, 20, 255);

  uint32_t color = strip.gamma32(strip.ColorHSV(hue, 255, brightness));
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, color);
  }
  strip.show();
  delay(20);
  checkButton();
}

```

```
// ----- Button handling with debounce -----
void checkButton() {
  int reading = digitalRead(BUTTON_PIN);

  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading == LOW && lastButtonState == HIGH) {
      mode = (mode + 1) % NUM_MODES;    // Move to the next mode
      strip.clear();
      strip.show();
      Serial.print("Mode changed to: ");
      Serial.println(mode);
    }
  }
  lastButtonState = reading;
}
```

Checks for understanding

- "What's the difference between using RGB colour and HSV colour in this code?"
- "Why do we call checkButton() inside the animation functions, not just once at the top of loop()?"
- "If your sound sensor reading barely changes when you shout, what would you adjust first?"

MISTAKE	FIX
Rainbow appears frozen or only shows one colour	Check <code>firstPixelHue</code> is declared <code>static</code> so it keeps its value between loop() calls
Heartbeat flickers instead of pulsing smoothly	Check <code>constrain()</code> is applied after adding heartStep, and delay isn't set to 0
Sound react mode stays one colour regardless of noise	Confirm sound sensor AO wire is on A0 not a digital pin, and check Serial Monitor values are actually changing
Button press skips two modes at once, or does nothing	Check debounceDelay logic wasn't deleted and that button legs go to D7 and GND, not 5V

Part 3 — Personalise: Finish the Wearable

Objectives

- Students permanently mount and decorate their wearable using craft materials.
- Students add a personalised "signature colour" mode to their code, editable with their own RGB values.

- Students prepare their finished wearable, running on power bank, for the showcase.

Timing (25 minutes)

- 0:00–0:05 — Introduce the craft station, decoration options, and glue-gun safety reminder
- 0:05–0:12 — Add the personalised "My Colour" mode to the sketch and upload
- 0:12–0:20 — Decorate wristband/cap, permanently secure wiring with heat shrink/tape, mount Nano and power bank in the pouch with Velcro
- 0:20–0:25 — Final function test on power bank (not laptop USB) and line up for showcase

Step-by-step

1. Explain that this final code change is entirely their own: they choose an RGB colour that means something to them (favourite colour, team colour, birthstone colour) and type it into three variables at the top of the sketch.
2. Walk students through adding the fourth mode and increasing `NUM_MODES` to 4 so the button cycles through it.
3. Upload and confirm the new mode appears in the button cycle.
4. Move to the craft table: demonstrate a clean glue line along the strip edge, and show how to loop excess wire and tuck it under a piece of felt rather than leaving it dangling.
5. Circulate as students decorate — felt shapes, sequins, painted initials, ribbon trim — reminding them the sound sensor mic and button must stay uncovered and accessible.
6. Help each student unplug the laptop USB cable and plug in their charged power bank instead; confirm the wearable still runs through all four modes on battery power.
7. Do a final walk-round check per student against the "done" checklist (see Assessment section) before showcase.

Build detail

Materials: felt scraps, sequins, ribbon, buttons, fabric paint pens, permanent markers, hot glue gun, heat-shrink tubing, Velcro coins, charged USB power bank.

Wiring/assembly list: no new electrical connections — this is the permanent mechanical finish. Apply glue only along strip edges and Nano housing, never over

the button or sound sensor mic hole; wrap any exposed joint in heat shrink before decorating over it; final power connection is power bank → Nano Micro-USB port (replacing the programming USB cable).

```
#include <Adafruit_NeoPixel.h>

#define LED_PIN      6
#define LED_COUNT    8
#define BUTTON_PIN   7
#define SOUND_PIN    A0

Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

// ==== PERSONALISE THIS! Type in your own favourite RGB colour (0-255 each) ====
int signatureRed    = 255;    // Example: hot pink = 255, 20, 147
int signatureGreen  = 0;
int signatureBlue   = 150;
// =====

int mode = 0;
const int NUM_MODES = 4;    // 0 Rainbow, 1 Heartbeat, 2 Sound React, 3 My Colour

int lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;

float heartBrightness = 0;
float heartStep = 6;

void setup() {
  strip.begin();
  strip.show();
  strip.setBrightness(70);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  checkButton();
  switch (mode) {
    case 0: rainbowCycle(10); break;
    case 1: heartbeatPulse(); break;
    case 2: soundReact(); break;
    case 3: myColor(); break;
  }
}

void rainbowCycle(int waitMs) {
  static long firstPixelHue = 0;
  for (int i = 0; i < strip.numPixels(); i++) {
    int pixelHue = firstPixelHue + (i * 65536L / strip.numPixels());
    strip.setPixelColor(i, strip.gamma32(strip.ColorHSV(pixelHue)));
  }
  strip.show();
  firstPixelHue += 256;
  if (firstPixelHue >= 65536) firstPixelHue = 0;
  delay(waitMs);
}
```

```

    checkButton();
}

void heartbeatPulse() {
    heartBrightness += heartStep;
    if (heartBrightness >= 255 || heartBrightness <= 0) {
        heartStep = -heartStep;
    }
    heartBrightness = constrain(heartBrightness, 0, 255);
    uint32_t red = strip.Color((int)heartBrightness, 0, 0);
    for (int i = 0; i < strip.numPixels(); i++) {
        strip.setPixelColor(i, red);
    }
    strip.show();
    delay(15);
    checkButton();
}

void soundReact() {
    int soundLevel = analogRead(SOUND_PIN);
    int hue = map(soundLevel, 0, 800, 43690, 0);
    hue = constrain(hue, 0, 65535);
    int brightness = map(soundLevel, 0, 800, 20, 255);
    brightness = constrain(brightness, 20, 255);
    uint32_t color = strip.gamma32(strip.ColorHSV(hue, 255, brightness));
    for (int i = 0; i < strip.numPixels(); i++) {
        strip.setPixelColor(i, color);
    }
    strip.show();
    delay(20);
    checkButton();
}

// ----- MODE 3: My Signature Colour (Personalise!) -----
void myColor() {
    uint32_t myColour = strip.Color(signatureRed, signatureGreen, signatureBlue);
    for (int i = 0; i < strip.numPixels(); i++) {
        strip.setPixelColor(i, myColour);
    }
    strip.show();
    checkButton();
}

void checkButton() {
    int reading = digitalRead(BUTTON_PIN);
    if (reading != lastButtonState) {
        lastDebounceTime = millis();
    }
    if ((millis() - lastDebounceTime) > debounceDelay) {
        if (reading == LOW && lastButtonState == HIGH) {
            mode = (mode + 1) % NUM_MODES;
            strip.clear();
            strip.show();
        }
    }
    lastButtonState = reading;
}

```

Checks for understanding

- "What line of code did you change to make mode 3 your own colour?"
- "Why do we need to increase NUM_MODES when we add a new mode?"
- "What could go wrong if you glue over the sound sensor's microphone hole?"

MISTAKE	FIX
New signature mode never appears when button pressed	Check NUM_MODES was changed from 3 to 4
Wearable worked on laptop USB but goes dark on power bank	Check power bank is charged and cable is a data/power cable, not a decorative-only cable
Sound react mode stops responding after decorating	Check glue hasn't covered the sensor's microphone opening
Wires pop loose during decorating	Always heat-shrink or tape a joint before gluing fabric over it, and avoid pulling wires taut while gluing

Assessment & Showcase

Assess each student against a simple four-point "done" checklist, checked live during the final ten minutes of Part 3: (1) all four modes cycle correctly with the button, (2) the sound-react mode visibly responds to clapping or talking, (3) the wearable runs independently on its power bank with no laptop attached, (4) the wearable is fully decorated with no exposed bare wire. A wearable meeting all four points is "showcase ready."

Run the showcase as a short runway/demo circle: students wear their finished piece, come to the front one at a time, briefly explain their design choices (colour scheme, decoration theme, why they picked their signature colour), and demonstrate all four modes live by pressing their button in front of the group. Facilitator times each demo to roughly 60–90 seconds so every student presents within the closing slot. Award light-hearted peer-voted categories rather than ranked scores — Most Colourful, Best Reaction to Sound, Most Creative Decoration, Cleanest Build — so every student is recognised for something specific.

Extension & Home Challenges

- **Easy:** Add a fifth mode that flashes the student's signature colour on and off like a strobe, using a simple delay-based blink instead of a solid fill.
- **Medium:** Wire a second button (or reuse the existing one with a long-press timer) to control overall brightness in steps (dim / medium / bright) independent of the current mode, so the wearable can be dimmed for indoor use and brightened outdoors.
- **Hard:** Combine the heartbeat and sound-react ideas: make the heartbeat pulse speed itself change based on ambient sound level (louder room = faster pulse), requiring the student to read the sound sensor inside the heartbeat function and use its value to adjust `heartStep` in real time.